

Proyecto Programación Avanzada — "Imperios en Guerra"

Realizado por:

Daniel Antonio Olivares Hinojosa (20252020016)

Johan Santiago Camargo Olmos (20252020034)

Docente: Nancy Yaneth Gélvez García

Programación Avanzada

Ingeniería de Sistemas

Universidad Distrital Francisco José de Caldas

Grupo 020-85

2026

Objetivos

Objetivo general:

Desarrollar una versión funcional del videojuego de estrategia en tiempo real (RTS) “Imperios en Guerra”, inspirado en Age of Empires, utilizando C# y el motor Unity, aplicando los principios fundamentales de la Programación Orientada a Objetos y los conceptos de Programación Avanzada, integrando una arquitectura Modelo-Vista-Controlador (MVC), concurrencia mediante hilos y Tasks, comunicación en red mediante sockets TCP y manejo de archivos para el registro de los diferentes acontecimientos de la partida.

Objetivos específicos:

- Aplicar los conceptos de encapsulamiento, herencia, polimorfismo y abstracción en la construcción de las diferentes entidades del videojuego.
- Diseñar la estructura del sistema utilizando el patrón arquitectónico Modelo-Vista-Controlador.
- Implementar una interfaz gráfica de escritorio utilizando C# y Unity.
- Utilizar hilos y Tasks para gestionar procesos concurrentes como la recolección de recursos, construcción de edificios, entrenamiento de unidades, movimiento y comunicación en red.
- Implementar mecanismos de sincronización para evitar condiciones de carrera al modificar recursos y estructuras compartidas.
- Implementar un sistema de recursos que permita administrar oro, madera, comida, hierro y piedra.
- Construir un mapa funcional que permita validar coordenadas, movimiento, construcción y rutas de las unidades.
- Implementar unidades militares y edificios con diferentes características y comportamientos.
- Implementar comunicación entre dos jugadores mediante sockets TCP.
- Registrar la configuración inicial, las acciones relevantes y el resultado final de la partida mediante archivos.
- Implementar diferentes condiciones de victoria y derrota.
- Desarrollar pruebas de escritorio que permitan verificar el funcionamiento de las secciones críticas del programa.
- Documentar la estructura del sistema mediante diagramas UML, diagramas de flujo y diagramas de secuencia.

La guía del proyecto establece como requisitos la aplicación de MVC, concurrencia mediante hilos y comunicación en red entre dos jugadores, además del manejo de archivos y una interfaz gráfica desarrollada en Unity.

Descripción

El proyecto consiste en la implementación del videojuego de estrategia en tiempo real “Imperios en Guerra” utilizando C# y el motor Unity, aplicando los conocimientos adquiridos durante el desarrollo de la asignatura Programación Avanzada.

El jugador controla un imperio dentro de un mapa compuesto por diferentes tipos de terreno y recursos. Cada jugador dispone de un Centro Urbano y puede administrar recursos, construir edificios, entrenar unidades, desplazar sus tropas y atacar las estructuras y unidades del oponente.

El objetivo principal consiste en desarrollar el imperio mediante una adecuada administración de los recursos disponibles y utilizar las unidades militares para derrotar al jugador contrario. La partida puede finalizar cuando el Centro Urbano del oponente es destruido o cuando se cumplen las condiciones de eliminación de sus unidades militares.

El proyecto maneja diferentes tipos de recursos, entre ellos oro, madera, comida, hierro y piedra. Estos recursos son utilizados para realizar las diferentes acciones disponibles durante la partida, como la construcción de edificios y el entrenamiento de unidades.

Dentro del sistema se implementaron diferentes tipos de unidades, entre ellas Aldeano, Soldado, Arquero y Caballero. Cada unidad posee características propias relacionadas con su vida, ataque, defensa, rango de ataque, posición y estado.

También se implementaron diferentes edificios, entre ellos Centro Urbano, Cuartel, Torre y Casa. Cada edificio posee características particulares y cumple una función específica dentro del desarrollo del imperio.

Para representar los diferentes elementos del videojuego se implementó una estructura orientada a objetos basada en clases independientes que representan conceptos como Jugador, Unidad, Edificio, Mapa, Recurso y Partida.

El proyecto incorpora dos modalidades principales de juego. En el modo PVE, el jugador se enfrenta a una inteligencia artificial que administra sus recursos y desarrolla su propio imperio. En el modo PVP, dos jugadores pueden competir utilizando una conexión de red mediante sockets TCP.

Adicionalmente, el sistema incorpora un mercado de intercambio de recursos y una herrería que permite realizar mejoras sobre las unidades. También se implementó un sistema de ítems que proporciona bonificaciones temporales a determinadas unidades.

Todo el sistema fue organizado siguiendo la arquitectura Modelo-Vista-Controlador, permitiendo separar la lógica del videojuego, la representación gráfica y la interacción del usuario.

La estructura solicitada por la guía establece que el Modelo debe estar compuesto por clases C# independientes de Unity, la Vista debe encargarse exclusivamente de la representación gráfica y el Controlador debe recibir las acciones del usuario, actualizar el Modelo y notificar a la Vista.

Explicación de la Arquitectura MVC

Para el desarrollo del proyecto se implementó el patrón arquitectónico Modelo-Vista-Controlador (MVC). Este patrón permite dividir el sistema en tres componentes principales, cada uno con responsabilidades específicas.

La principal ventaja de MVC consiste en separar la lógica del sistema de la interfaz gráfica y del manejo de las acciones realizadas por el usuario. Esta separación permite organizar mejor el código y facilita el mantenimiento y ampliación del proyecto.

Dentro del videojuego “Imperios en Guerra”, la arquitectura MVC permite que la lógica principal del juego permanezca independiente de Unity. De esta manera, el Modelo administra las reglas del videojuego, la Vista representa gráficamente el estado actual y el Controlador funciona como intermediario entre ambos.

Cuando el usuario realiza una acción, por ejemplo seleccionar una unidad y hacer clic sobre una posición del mapa, la Vista detecta la interacción y el Controlador interpreta la acción. Posteriormente, el Controlador solicita al Modelo realizar el movimiento. El Modelo valida que la acción sea posible y actualiza su estado. Finalmente, la Vista representa visualmente el resultado.

Modelo

El Modelo comprende el núcleo principal del proyecto, ya que en esta capa se encuentra la lógica del videojuego, las reglas, los datos y el comportamiento de las diferentes entidades.

El Modelo administra las posiciones de las unidades y edificios, los recursos de cada jugador, el estado de la partida, el movimiento de las unidades, la construcción, el entrenamiento, los ataques, las condiciones de victoria y los diferentes procesos concurrentes.

Dentro de esta capa se implementaron las siguientes clases:

Simulación:

Es la clase principal del Modelo y actúa como motor del mundo del videojuego. Su función consiste en administrar el estado completo de la partida, incluyendo el mapa, los jugadores, las unidades, los edificios, los recursos y los items.

La clase también coordina diferentes Tasks encargadas de ejecutar procesos independientes como el reloj de la partida, entrenamiento de unidades, construcción de edificios, recolección de recursos, generación de items, expiración de efectos temporales, batalla e inteligencia artificial.

Además, administra el mecanismo de sincronización mediante un objeto de bloqueo para proteger los datos compartidos y evitar que diferentes tareas modifiquen simultáneamente el mismo recurso o estructura.

Jugador:

Representa a cada uno de los jugadores que participan en la partida.

La clase administra los recursos disponibles, las unidades, los edificios y las mejoras obtenidas durante la partida.

Cada jugador puede administrar cinco tipos de recursos:

- Oro.
- Madera.
- Comida.
- Hierro.
- Piedra.

También posee métodos para verificar si puede pagar una determinada acción, gastar recursos, recibir recursos y determinar si se encuentra derrotado.

Unidad:

Representa las entidades móviles del juego.

Cada unidad posee información relacionada con su tipo, vida, ataque, defensa, rango de ataque, posición y estado actual.

Entre los estados que puede presentar una unidad se encuentran:

- Idle.
- Moviendo.
- Recolectando.
- Construyendo.
- Atacando.

Las unidades pueden desplazarse por el mapa, recolectar recursos, construir edificios y participar en enfrentamientos contra unidades enemigas.

Edificio:

Representa las estructuras construibles dentro del mapa.

Entre los diferentes edificios se encuentran:

- Centro Urbano.
- Cuartel.
- Torre.
- Casa.

Cada edificio posee una posición, vida, tamaño, costo de construcción y estado.

Los estados principales de los edificios son:

- EnConstruccion.
- Operativo.
- Destruído.

Los edificios cumplen funciones diferentes dentro del desarrollo del imperio. Por ejemplo, el Cuartel permite entrenar unidades militares, mientras que la Torre puede utilizarse como estructura defensiva.

Mapa:

Representa el tablero donde se desarrolla la partida.

El mapa se implementa mediante una matriz de 140×140 casillas y permite validar diferentes condiciones relacionadas con las posiciones.

Entre las operaciones realizadas por esta clase se encuentran:

- Validar coordenadas.
- Verificar si una casilla está libre.
- Verificar si una posición es transitable.
- Verificar si una posición permite construir.
- Determinar rutas para las unidades.

También se implementó un algoritmo de búsqueda BFS para realizar el pathfinding de las unidades.

La guía establece que el mapa debe validar coordenadas, impedir posiciones inválidas y permitir seleccionar destinos directamente sobre la representación gráfica.

Recurso:

Representa los diferentes yacimientos disponibles dentro del mapa.

Cada recurso posee un tipo, una cantidad y una posición.

Los principales tipos de recursos son oro, madera, comida, hierro y piedra.

Los aldeanos pueden desplazarse hacia estos recursos y realizar actividades de recolección para aumentar la cantidad disponible para el jugador.

Partida:

Actúa como coordinador general del estado de la partida.

Administra información relacionada con el tiempo de juego, el estado de ejecución, el ganador y el motivo por el cual finalizó la partida.

También permite determinar cuándo debe finalizar el juego y cuándo se debe mostrar el resultado correspondiente.

ConectorRed:

Gestiona la comunicación entre los dos jugadores mediante sockets TCP.

Su función consiste en establecer una conexión, enviar acciones al oponente, recibir mensajes y manejar situaciones relacionadas con la pérdida de conexión.

El sistema utiliza un hilo independiente para escuchar los mensajes provenientes de la red, evitando que la espera de información bloquee la interfaz gráfica.

La guía exige que la comunicación entre las dos instancias del juego se realice mediante una tecnología de red y que el hilo encargado de escuchar los mensajes no bloquee el hilo principal de Unity.

GestorArchivos:

Es la clase encargada de administrar los archivos generados por el videojuego.

El sistema utiliza archivos para almacenar:

- configuracion.txt.
- log_partida.txt.
- resultado_final.txt.

El archivo de configuración permite almacenar el estado inicial de la partida.

El archivo de logs registra las acciones relevantes realizadas durante el juego.

El archivo de resultado final almacena la información relacionada con el estado final y el ganador.

La guía establece específicamente estos tres archivos como parte de los entregables del proyecto.

IAEnemiga:

Implementa el comportamiento de la inteligencia artificial utilizada en el modo PVE.

La IA puede realizar diferentes acciones de manera autónoma, como recolectar recursos, construir edificios y entrenar unidades.

Su ejecución se realiza mediante una Task independiente para evitar bloquear la ejecución principal del videojuego.

DatosDelJuego:

Funciona como catálogo central de información relacionada con las unidades, edificios y recursos.

Contiene los costos y estadísticas base utilizados para crear las diferentes entidades.

De esta forma se evita duplicar información en diferentes partes del código.

InstantaneaJuego:

Representa una copia del estado actual del mundo.

La Vista utiliza esta información para representar el estado de la partida sin acceder directamente a las estructuras internas que están siendo modificadas por los procesos concurrentes.

Esta separación ayuda a evitar problemas relacionados con la lectura simultánea de datos que pueden estar siendo modificados por otros hilos.

Tipos:

Contiene las enumeraciones utilizadas por el sistema para representar diferentes tipos y estados.

Entre ellas se encuentran:

- TipoUnidad.
- TipoEdificio.
- TipoRecurso.
- TipoItem.
- EstadoUnidad.
- EstadoEdificio.
- RitmoPartida.
- ModoVictoria.

Vista

La Vista corresponde a la representación gráfica del sistema.

Su función consiste en mostrar visualmente la información almacenada en el Modelo y presentar al usuario los diferentes elementos del videojuego.

La Vista no contiene las reglas principales del juego ni modifica directamente el Modelo. Su responsabilidad consiste en representar el estado actual y recibir las interacciones del usuario.

El proyecto utiliza Unity y su sistema de interfaz gráfica para construir la representación visual del videojuego.

GestorJuego:

Actúa como raíz de la Vista y coordina los diferentes componentes visuales.

También se encarga de iniciar el Controlador y ejecutar el ciclo de actualización de Unity.

VistaTablero:

Representa gráficamente el mapa y las entidades existentes dentro de la partida.

Muestra las unidades, edificios, recursos y demás elementos del escenario.

También puede representar las barras de vida y los diferentes estados visuales de las entidades.

ControllInputUsuario:

Se encarga de recibir las acciones realizadas mediante el mouse y el teclado.

Permite seleccionar unidades, seleccionar posiciones del mapa y activar las diferentes acciones disponibles.

También administra funcionalidades propias de un juego RTS como selección por arrastre, grupos de control y desplazamiento de la cámara.

HudRecursos:

Muestra permanentemente información relacionada con los recursos disponibles para el jugador.

Entre los datos mostrados se encuentran:

- Oro.
- Madera.
- Comida.
- Hierro.
- Piedra.
- Tiempo de juego.
- Estado de la partida.
- Mensajes relacionados con las acciones.

La guía establece que la interfaz debe mostrar dinámicamente el estado del mapa, recursos, unidades y mensajes del juego.

MenuInicio:

Presenta las opciones iniciales del videojuego.

Desde este menú el usuario puede seleccionar el modo de juego y las diferentes configuraciones disponibles.

Entre las opciones implementadas se encuentran:

- PVE.
- PVP.
- Selección de bases enemigas.
- Ritmo de partida.
- Inicio con recursos.
- Modo exploración.

MenuMercado:

Implementa el sistema de intercambio de recursos.

Permite realizar operaciones de compra y venta utilizando oro como recurso de intercambio.

MenuMejoras:

Representa la herrería del videojuego.

Permite mejorar características de las unidades relacionadas con:

- Ataque.
- Defensa.
- Recolección.

Cada característica puede ser mejorada mediante diferentes niveles.

Minimap:

Representa una versión reducida del mapa principal.

Muestra las posiciones generales de las tropas, edificios y recursos y permite identificar visualmente la zona del mapa que está siendo observada por el jugador.

PanelFinPartida:

Muestra el resultado final de la partida.

Cuando se cumple una condición de victoria o derrota, el panel informa al usuario el resultado y presenta las estadísticas correspondientes.

SonidoJuego:

Administra los diferentes sonidos utilizados por el videojuego.

Puede producir sonidos relacionados con acciones como:

- Selección.
- Errores.
- Ataques.
- Construcción.
- Entrenamiento.
- Victoria.
- Derrota.

UiFabrica:

Proporciona elementos reutilizables de interfaz como paneles, textos y botones.

Su utilización permite mantener una estructura visual uniforme dentro del videojuego.

ArteRecursos:

Administra la carga de los recursos gráficos utilizados en el videojuego.

Permite cargar sprites relacionados con edificios, unidades, recursos, terreno e interfaz.

SpriteFactory:

Permite generar representaciones gráficas alternativas cuando no se encuentra disponible un recurso gráfico específico.

Información mostrada en pantalla:

La Vista muestra permanentemente:

- Recursos disponibles.
- Tiempo de juego.
- Estado de la partida.
- Mensajes al usuario.
- Unidades.
- Edificios.
- Recursos del mapa.
- Minimapa.

Adicionalmente presenta pantallas de:

- Inicio.
- PVE.
- PVP.
- Mercado.
- Herrería.
- Victoria.
- Derrota.

Controlador

El Controlador funciona como intermediario entre el Modelo y la Vista, transformando las acciones del usuario en operaciones sobre la lógica del videojuego.

Por ejemplo, cuando el usuario selecciona una unidad y hace clic sobre una posición del mapa, el Controlador recibe la acción y solicita al Modelo realizar el movimiento.

El Modelo valida las coordenadas, determina si el movimiento es posible y actualiza la posición de la unidad. Posteriormente, la Vista obtiene el nuevo estado y representa la unidad en su nueva posición.

El Controlador también administra acciones como:

- Mover unidades.
- Construir edificios.
- Entrenar unidades.
- Recolectar recursos.
- Atacar.
- Comprar y vender recursos.
- Aplicar mejoras.
- Enviar acciones mediante la red.
- Procesar mensajes recibidos.
- Registrar acciones en los archivos.

Una característica importante es que el Controlador no administra directamente la concurrencia. Los procesos concurrentes pertenecen al Modelo, mientras que el Controlador se encarga de solicitar operaciones y comunicar los resultados entre las diferentes capas.

La separación entre las tres capas corresponde a uno de los requisitos fundamentales establecidos para el proyecto.

Explicación de Concurrencia

La concurrencia constituye uno de los componentes principales del proyecto de Programación Avanzada.

Su implementación permite que diferentes procesos del videojuego se ejecuten simultáneamente sin bloquear la interfaz gráfica ni el funcionamiento general de la partida.

La clase Simulacion utiliza diferentes Tasks para administrar procesos independientes del juego.

Entre estos procesos se encuentran:

- Reloj de la partida.
- Entrenamiento de unidades.
- Construcción de edificios.
- Recolección de recursos.
- Generación de items.
- Expiración de efectos temporales.
- Bucle de batalla.
- Inteligencia artificial.

Cada proceso realiza una tarea específica y puede ejecutarse de forma independiente.

La guía exige que la concurrencia se utilice para actividades como recolección, construcción, entrenamiento, movimiento y escucha de mensajes de red. También establece que los hilos secundarios no deben modificar directamente componentes de Unity.

Concurrencia en la recolección

La recolección de recursos se ejecuta mediante tareas independientes.

Cuando un aldeano se encuentra realizando una actividad de recolección, el proceso correspondiente administra el tiempo necesario y posteriormente actualiza la cantidad del recurso del jugador.

Esto permite que mientras un aldeano recolecta recursos, el jugador pueda realizar otras acciones como construir edificios o mover unidades.

Concurrencia en la construcción

La construcción de edificios también posee un proceso independiente.

Cuando el jugador inicia una construcción, el sistema valida los recursos disponibles, descuenta el costo correspondiente y comienza el tiempo de construcción.

Mientras el edificio se encuentra en construcción, las demás acciones de la partida pueden continuar ejecutándose.

Concurrencia en el entrenamiento

Los edificios que permiten crear unidades poseen procesos encargados de controlar el tiempo de entrenamiento.

Una vez terminado el proceso, la unidad correspondiente es creada y agregada al jugador.

Concurrencia en la batalla

Las acciones relacionadas con los ataques se procesan dentro del Modelo.

El sistema verifica las posiciones, el rango de ataque, la vida de las unidades y el daño correspondiente.

La sincronización permite que diferentes acciones no modifiquen simultáneamente información de una misma unidad de manera inconsistente.

Concurrencia en la inteligencia artificial

La IA funciona mediante una Task independiente.

Mientras el jugador humano realiza sus acciones, la IA analiza el estado de su imperio y toma decisiones relacionadas con la recolección, construcción y entrenamiento.

De esta manera, el modo PVE mantiene un comportamiento autónomo del oponente.

Concurrencia en la comunicación de red

El componente de red utiliza un hilo independiente encargado de escuchar los mensajes recibidos mediante TCP.

Este hilo permanece separado de la interfaz gráfica para evitar que una espera de comunicación bloquee Unity.

Cuando se recibe un mensaje, este es almacenado para posteriormente ser procesado por el sistema.

La guía especifica que el hilo de escucha de red debe funcionar sin bloquear la interfaz gráfica ni el hilo principal de Unity.

Concurrencia en los archivos

La escritura de archivos se administra mediante un proceso independiente.

Las acciones que deben ser registradas son almacenadas en una cola y posteriormente un hilo escritor se encarga de escribirlas en el archivo correspondiente.

Esto evita que una operación de escritura en disco detenga las acciones del juego.

Sincronización

Debido a que diferentes procesos pueden acceder simultáneamente a los recursos y estructuras del juego, es necesario utilizar mecanismos de sincronización.

El sistema utiliza lock y estructuras concurrentes para proteger los datos compartidos.

Por ejemplo, si dos procesos intentaran modificar simultáneamente la cantidad de oro de un jugador, podría producirse una condición de carrera.

Por esta razón, las operaciones críticas se realizan dentro de secciones sincronizadas.

Una regla fundamental utilizada en el proyecto consiste en no realizar operaciones de escritura de red o disco dentro del bloqueo principal del Modelo.

Los mensajes de red se almacenan en colas y posteriormente son enviados fuera de la sección crítica. De igual manera, los registros de archivos se colocan en una cola para que sean procesados por el hilo escritor.

La guía establece explícitamente la necesidad de utilizar mecanismos como lock, Monitor o estructuras de `System.Collections.Concurrent` para evitar condiciones de carrera.

Elementos de concurrencia

Elemento		Tipo	Propósito
Reloj	Task		Controlar el tiempo de la partida
Recolección	Task		Gestionar la obtención de recursos
Construcción	Task		Controlar el tiempo de construcción
Entrenamiento	Task		Controlar la creación de unidades
Items	Task		Generar elementos especiales
Expiración	Task		Controlar efectos temporales
Batalla	Task		Gestionar procesos de combate
IA	Task		Ejecutar las decisiones del enemigo
Escucha TCP	Thread		Recibir mensajes de red

Escritura de logs	Thread	Registrar eventos en disco
Sincronización	lock	Proteger información compartida
Colas	ConcurrentQueue / BlockingCollection	Intercambiar información entre procesos

Comunicación en red

El modo PVP utiliza sockets TCP para permitir la comunicación entre dos instancias del videojuego.

Una instancia puede actuar como servidor y esperar la conexión del segundo jugador, mientras que la otra instancia funciona como cliente y establece la conexión.

Una vez establecida la conexión, los jugadores pueden intercambiar mensajes relacionados con las acciones realizadas durante la partida.

Entre las acciones que pueden transmitirse se encuentran:

- Movimiento.
- Construcción.
- Entrenamiento.
- Ataque.
- Otras acciones relevantes del juego.

Los mensajes utilizan una estructura definida que permite identificar el tipo de acción y los datos necesarios para ejecutarla.

Por ejemplo, un mensaje relacionado con el movimiento puede contener el tipo de acción y las coordenadas de destino.

La recepción de mensajes se realiza mediante un hilo independiente.

Cuando se pierde la conexión, el sistema debe detectar el problema y gestionar el estado correspondiente.

La guía exige que las acciones realizadas por un jugador sean transmitidas al oponente y que se manejen adecuadamente las pérdidas de conexión y los errores de comunicación.

Manejo de archivos

El proyecto utiliza System.IO para almacenar información importante relacionada con la partida.

Configuracion.txt

Este archivo almacena la configuración inicial del videojuego.

Entre la información que puede registrarse se encuentra:

- Configuración del mapa.
- Posiciones iniciales.
- Recursos iniciales.
- Configuración de los jugadores.
- Parámetros seleccionados para la partida.

La configuración inicial debe almacenarse antes de comenzar la partida para permitir revisar posteriormente las condiciones con las que inició el juego.

Log_partida.txt

Este archivo almacena los acontecimientos relevantes de la partida.

Un registro puede contener información como:

Turno: Jugador 1

Acción: Ataque

Resultado: Impacto - Unidad enemiga destruida

También pueden registrarse acciones de movimiento, construcción, entrenamiento, recolección y otras operaciones importantes.

Resultado_final.txt

Al terminar la partida, este archivo almacena información relacionada con el resultado final.

Entre los datos registrados se encuentran:

- Estado final del mapa.
- Jugador ganador.
- Motivo de la victoria.
- Estado final de las unidades principales.

La documentación del proyecto establece específicamente que se deben guardar la configuración inicial, los eventos relevantes y el resultado final en archivos.

Aplicación de los Principios de Programación Orientada a Objetos

Encapsulamiento

El encapsulamiento consiste en proteger los datos internos de las clases y permitir su manipulación mediante métodos definidos.

Por ejemplo, la clase Jugador administra internamente sus recursos y proporciona métodos que permiten verificar si una determinada acción puede realizarse.

```
public class Jugador
```

```

{
    private string nombre;
    private Dictionary<string, int> recursos;
    private List<Unidad> unidades;
    private List<Edificio> edificios;

    public bool PuedePagar(int madera, int oro, int comida)
    {
        return recursos["Madera"] >= madera &&
            recursos["Oro"] >= oro &&
            recursos["Comida"] >= comida;
    }
}

```

De esta forma, las operaciones relacionadas con los recursos permanecen controladas por la propia clase.

Herencia

La herencia permite reutilizar características comunes entre diferentes clases.

Por ejemplo, las unidades pueden compartir características generales relacionadas con su posición, vida, ataque y defensa, mientras que los diferentes tipos de unidades pueden especializar su comportamiento.

De igual manera, los diferentes tipos de edificios pueden compartir características generales relacionadas con su posición, vida y estado.

Esta estructura permite evitar la duplicación de código y organizar mejor las entidades del videojuego.

Polimorfismo

El polimorfismo permite trabajar con diferentes objetos mediante una misma interfaz o clase base.

Por ejemplo, las diferentes unidades pueden ser tratadas como objetos de tipo Unidad, aunque cada una posea características específicas.

Un Soldado, un Arquero y un Caballero pueden utilizar operaciones generales de una unidad, pero sus estadísticas y comportamiento pueden ser diferentes.

De esta forma, el código puede trabajar con la clase general sin necesidad de conocer todos los detalles de cada tipo específico.

Abstracción

La abstracción permite representar conceptos generales mediante clases que contienen las características esenciales de las entidades.

Por ejemplo, una clase abstracta puede representar una entidad general del videojuego:

```
public abstract class Entidad
{
    public int PosicionX { get; set; }
    public int PosicionY { get; set; }
    public bool EstaViva { get; set; }
}
```

Esta clase representa las características que pueden compartir diferentes elementos del videojuego.

Interfaces

Las interfaces permiten establecer comportamientos comunes sin obligar a que las clases tengan la misma implementación interna.

En el proyecto pueden utilizarse interfaces para definir comportamientos relacionados con entidades del juego, movimiento, ataque o acciones que diferentes objetos puedan realizar.

Esto facilita la extensión del sistema y permite aplicar polimorfismo mediante contratos comunes.

Colecciones

El proyecto utiliza diferentes estructuras de datos para administrar los elementos dinámicos del juego.

Entre ellas se encuentran:

- List<T> para almacenar unidades y edificios.
- Dictionary<TKey,TValue> para administrar recursos y estadísticas.
- ConcurrentQueue<T> para manejar mensajes entre procesos.
- BlockingCollection<T> para administrar los eventos que deben ser escritos en archivos.

La utilización de colecciones permite organizar y manipular eficientemente los elementos de la partida.

La guía solicita explícitamente el uso de colecciones como List<T> y Dictionary<TKey,TValue> para administrar unidades, edificios y otros elementos.

Manejo de excepciones

El proyecto utiliza mecanismos try-catch para controlar errores que puedan producirse durante la ejecución.

Este mecanismo es especialmente importante en procesos como:

- Comunicación de red.

- Lectura y escritura de archivos.
- Inicialización de la partida.
- Procesamiento de mensajes.
- Operaciones relacionadas con las acciones del jugador.

El objetivo consiste en evitar que un error aislado provoque el cierre inesperado del videojuego.

La guía establece como requisito el manejo de excepciones mediante try-catch.

Validación de acciones

Antes de ejecutar una acción, el sistema verifica que las condiciones necesarias se cumplan.

Por ejemplo, antes de construir un edificio se comprueba:

- Que la posición esté dentro del mapa.
- Que la posición esté disponible.
- Que exista una unidad capaz de construir.
- Que el jugador tenga recursos suficientes.
- Que el edificio no se sobreponga con otro elemento.

Para el movimiento se verifica:

- Que las coordenadas sean válidas.
- Que la posición sea transitable.
- Que exista una unidad seleccionada.
- Que el movimiento solicitado sea posible.

Para atacar se verifica:

- Que exista una unidad atacante.
- Que el objetivo exista.
- Que el atacante esté dentro del rango permitido.
- Que la unidad pueda atacar.
- Que el objetivo se encuentre activo.

Estas validaciones permiten evitar estados inválidos dentro del videojuego.

Conclusiones

El desarrollo del proyecto permitió aplicar de forma práctica los conceptos fundamentales de la Programación Orientada a Objetos mediante la construcción de un videojuego de estrategia en tiempo real basado en Age of Empires. A través de clases, encapsulamiento, herencia, polimorfismo, abstracción e interfaces se logró organizar la estructura interna del sistema y representar las diferentes entidades que participan dentro de la partida.

La implementación de la arquitectura Modelo-Vista-Controlador permitió separar adecuadamente la lógica del videojuego, la representación gráfica y el manejo de las acciones realizadas por el usuario. Esta separación facilita la organización del proyecto y permite modificar componentes de una capa sin afectar directamente las responsabilidades de las demás.

El uso de concurrencia mediante Tasks y Threads permitió ejecutar de manera independiente procesos como la recolección de recursos, construcción de edificios, entrenamiento de unidades, inteligencia artificial, comunicación de red y escritura de archivos. La sincronización mediante mecanismos como lock y estructuras concurrentes permitió controlar el acceso a los datos compartidos y reducir la posibilidad de condiciones de carrera.

La implementación de comunicación mediante sockets TCP permitió incorporar el componente de programación en red solicitado para el proyecto, haciendo posible que dos jugadores intercambien acciones durante una partida. Además, el uso de un hilo independiente para escuchar mensajes evita que la comunicación bloquee la interfaz principal del videojuego.

El manejo de archivos permitió registrar la configuración inicial, las acciones importantes y el resultado final de la partida. Esto proporciona información que puede utilizarse posteriormente para revisar el desarrollo de una partida y verificar los resultados obtenidos.

La implementación de diferentes unidades, edificios, recursos, mejoras, mercado, inteligencia artificial y condiciones de victoria permitió construir un sistema con diferentes mecánicas de juego y una estructura suficientemente modular para continuar ampliándolo.

Finalmente, el desarrollo del proyecto permitió integrar los conocimientos de Programación Orientada a Objetos con conceptos propios de Programación Avanzada como concurrencia, sincronización, comunicación en red, manejo de archivos, estructuras de datos y arquitectura MVC. De esta manera se obtuvo un sistema organizado y orientado a cumplir los requerimientos establecidos para el proyecto de la asignatura.